

Relatório do Trabalho: Sistema de Locadora de Filmes

Gustavo Antonio de Sousa, Luan Cristan da Silva Rufino, Rian de Oliveira Leal

1

²Departamento de Sistemas de Informação
Universidade Federal do Piauí, Picos, Piauí, Brasil

`gustavo.antonio@ufpi.edu.br, luan.rufino@ufpi.edu.br, rian.leal@ufpi.edu.br`

Abstract. *This report describes the development of a movie rental graphical interface built with Python and PySide6. The project fulfills all the technical requirements specified by the professor, including the implementation of signals, slots, events, multiple widgets, layouts, toolbars, menus, and secondary windows. The document outlines the construction strategy, the widget documentation, and the specific use of Generative AI during the development process.*

Resumo. *Este documento apresenta o relatório do desenvolvimento do sistema de Locadora de Filmes. O grupo construiu o projeto usando a linguagem Python e a biblioteca PySide6. O texto descreve como todos os requisitos exigidos foram feitos com sucesso.*

1. Informações Gerais e Descrição do Projeto

O projeto consiste em uma interface gráfica criada para gerenciar um catálogo de obras de uma locadora. Criamos funcionalidades operacionais para listar as obras, cadastrar novos títulos, alugar, devolver e excluir filmes do catálogo.

A interface gráfica foi construída inteiramente com a biblioteca PySide6, respeitando o tema central de locadora de filmes. O código fonte foi organizado em arquivos separados para facilitar a leitura e a manutenção do sistema.

2. Como os Requisitos Foram Atendidos

Abaixo, explicamos como cada um dos requisitos técnicos exigidos para a nota foi implementado na prática dentro do código.

2.1. Sinais e Slots

O sistema possui diversas conexões nativas de sinais e slots, como as respostas aos cliques dos botões da interface. Além dos sinais comuns, o grupo criou um sinal customizado chamado `statusModificado` na janela de detalhes do filme. Esse sinal avisa a janela principal para atualizar a tabela visual logo após o usuário registrar a locação ou a devolução de uma obra através da janela secundária.

2.2. Eventos

O grupo reescreveu os eventos virtuais nativos da janela principal para melhorar a navegação. Nós implementamos o evento de fechamento (`closeEvent`) para exibir uma caixa de pergunta antes de encerrar o programa de forma definitiva. Nós também alteramos o evento de teclado (`keyPressEvent`) para o programa abrir a tela de detalhes do filme automaticamente quando o usuário seleciona uma linha da tabela e aperta a tecla Enter.

2.3. Documentação de Todos os Widgets Criados

Para cumprir a exigência de documentar todos os widgets, o grupo elaborou a lista detalhada abaixo contendo os nomes das variáveis instanciadas, o tipo do componente e a respectiva função na interface:

Na Janela Principal (`janela_principal.py`):

- `campo_busca_rapida` (QLineEdit): recebe o texto para busca rápida na barra de ferramentas.
- `status_bar` (QStatusBar): exibe as mensagens temporárias de operação no rodapé.
- `layout_pilha` (QStackedLayout): gerencia a alternância das páginas centrais.
- `campo_filtro_titulo` (QLineEdit): recebe o texto para filtrar os filmes pelo título.
- `combo_filtro_genero` (QComboBox): seleciona o gênero para filtrar a lista.
- `tabela_filmes` (QTableWidget): exibe o catálogo completo de obras em formato de tabela.
- `barra_progresso` (QProgressBar): indica visualmente a porcentagem de filmes disponíveis para locação.
- `btn_locar` (QPushButton): aciona a locação ou a devolução do item selecionado na tabela.

No Diálogo de Cadastro (`dialogo_cadastro.py`):

- `campo_titulo` (QLineEdit): captura o título do novo filme.
- `combo_genero` (QComboBox): lista os gêneros disponíveis para escolha.
- `spin_ano` (QSpinBox): captura o ano de lançamento variando de 1900 a 2030.
- `spin_preco` (QDoubleSpinBox): captura o preço da diária do filme.
- `radio_dvd`, `radio_bluray`, `radio_digital` (QRadioButton): definem o tipo de mídia de forma mutuamente exclusiva através de um `QButtonGroup`.
- `check_disponivel` (QCheckBox): marca o status inicial de disponibilidade.
- `slider_avaliacao` (QSlider): barra deslizante para o usuário atribuir uma nota inicial.
- `label_valor_slider` (QLabel): exibe o valor numérico exato do slider na tela.
- `texto_sinopse` (QTextEdit): captura a descrição longa ou sinopse da obra.
- `caixa_botoes` (QDialogButtonBox): agrupa os botões padrão de confirmação (Ok) e cancelamento (Cancel).

Na Janela de Detalhes (`janela_detalhes.py`):

- `grupo_info` (QGroupBox): agrupa visualmente as informações da ficha técnica do filme.
- `label_titulo`, `label_genero`, `label_ano`, `label_preco`, `label_midia`, `label_status`, `label_avaliacao`, `label_sinopse` (QLabel): conjunto de rótulos dedicados para exibir os dados do filme em modo de leitura.
- `botao_alternar_status` (QPushButton): botão dedicado para realizar a locação ou devolução diretamente pelos detalhes.
- `botao_fechar` (QPushButton): botão simples para ocultar a janela secundária.

2.4. Layouts

A organização visual da tela foi feita combinando vários gerenciadores de geometria. Nós empilhamos as páginas da tela central com o gerenciador `QStackedLayout`. Para alinhar os componentes básicos, nós adotamos o `QVBoxLayout` para criar listas verticais e o `QHBoxLayout` para estruturar as linhas horizontais. O formulário de cadastro de novos filmes utiliza uma grade invisível (`QGridLayout`) para manter os rótulos de texto e os campos de entrada perfeitamente alinhados lado a lado.

2.5. Barras de Ferramentas e Menu

A tela principal exibe uma barra de menu (`QMenuBar`) no topo que abriga a opção de ajuda e os créditos do sistema. Logo abaixo do menu, nós implementamos uma barra de ferramentas fixa (`QToolBar`). Essa barra contém os atalhos para as ações principais do programa, que servem para cadastrar, ver detalhes e excluir filmes. Todas essas funções foram criadas utilizando a classe `QAction`. Nós também colocamos um campo de texto direto na barra de ferramentas para o usuário fazer uma busca rápida de filmes.

2.6. Dialogs e Alertas

O projeto mostra vários alertas nativos (`QMessageBox`) na tela para avisar o usuário sobre regras de preenchimento, confirmar ações que deram certo e fazer perguntas sensíveis antes de excluir registros. Além dos alertas básicos, nós construímos uma janela de diálogo dedicada (`QDialog`) exclusiva para o cadastro de novos filmes. Essa janela bloqueia a tela principal até o usuário terminar de preencher os dados ou clicar no botão de cancelar a operação.

2.7. Pelo Menos Uma Janela Adicional

Para cumprir a exigência de ter uma tela extra independente, o sistema possui uma janela secundária batizada de `JanelaDetalhesFilme`. Ela herda da classe mãe `QWidget` e mostra a ficha técnica completa do filme selecionado. Essa tela funciona de forma totalmente solta, o que permite ao usuário interagir com o fundo da tela principal enquanto os detalhes do filme continuam visíveis na frente.

3. Uso de Inteligência Artificial

A Inteligência Artificial generativa atuou de forma estritamente controlada neste projeto. O grupo utilizou a ferramenta Gemini apenas como auxílio pontual para dois fins específicos autorizados pelas regras do trabalho. Nós garantimos enfaticamente que nenhum trecho da interface completa foi gerado pela IA, o que deixa inequívoco o cumprimento da regra do enunciado onde interfaces prontas e extremamente complexas não contam como auxílio.

Primeiramente, nós pedimos para a IA gerar as informações de exemplo para preencher a nossa base de dados inicial. Ela buscou na internet e forneceu as descrições reais e os anos de lançamento corretos dos filmes de exemplo do código: *Matrix*, *Interestelar* e *O Poderoso Chefão*.

Em segundo lugar, a IA foi consultada unicamente para tirar dúvidas de sintaxe da linguagem `PySide6` em três partes pontuais onde o código apresentou uma lógica um pouco mais complicada:

1. **Dúvida de layout da tabela:** Nós perguntamos para a IA como fazer a coluna de título da tabela expandir para ocupar todo o espaço vazio restante da tela. A ferramenta mostrou a sintaxe correta usando a propriedade `Stretch` diretamente no modo de redimensionamento do cabeçalho horizontal da tabela.
2. **Dúvida de agrupamento de botões:** Nós consultamos a IA para entender a sintaxe exata de como agrupar vários botões de rádio na mesma seleção lógica utilizando a classe `QButtonGroup`, e como extrair qual botão específico estava marcado no momento do cadastro.
3. **Dúvida de união de botões no alerta:** Nós utilizamos a IA para esclarecer a sintaxe de união dos botões padrão dentro de uma caixa de pergunta. A ferramenta mostrou como usar o operador de barra vertical (bitwise OR) para juntar as opções de botão de Sim e botão de Não na mesma caixa de mensagem.

4. Estrutura, Execução e Considerações Finais

4.1. Estrutura do Código

A divisão do projeto ficou configurada em quatro partes principais: a pasta "componentes" abriga as lógicas do diálogo de cadastro e da janela de detalhes; a pasta "dados" guarda o repositório responsável pelo armazenamento das informações dos filmes; a pasta "janelas" guarda a janela principal do sistema; e o arquivo "main.py" atua como o ponto de partida de toda a aplicação.

4.2. Instruções de Execução

Para iniciar a interface de forma correta, o usuário precisa garantir a instalação da linguagem Python na máquina. No sistema Linux, o primeiro passo é criar um ambiente virtual digitando "python3 -m venv venv" no terminal. Em seguida, o usuário ativa o ambiente usando "source venv/bin/activate". Com o ambiente ativo, é necessário baixar a biblioteca gráfica executando o comando "pip install pyside6". Por fim, basta digitar "python3 main.py" para o sistema abrir.

Para o sistema Windows, o usuário roda o projeto digitando o comando "python main.py" no terminal da máquina.

4.3. Considerações Finais

O código atingiu os objetivos propostos para o trabalho prático porque o grupo implementou todas as exigências da disciplina de forma plenamente funcional. A estruturação em pastas separadas manteve as responsabilidades bem divididas, o que garantiu uma leitura mais simples do código final e proporcionou um excelente exercício para fixar os conceitos de Programação Orientada a Objetos aliados ao design de interfaces.

Referências

- [1] THE QT COMPANY. *Qt for Python Documentation (PySide6)*. Oslo: The Qt Company, 2026. Disponível em: <https://doc.qt.io/qtforpython-6/>.
- [2] FITZPATRICK, Martin. *PySide6 Tutorials: Create GUI Applications with Python & Qt6*. [S.l.]: Python GUIs, 2026. Disponível em: <https://www.pythonguis.com/>.
- [3] EVANDRO, Professor. *Notas de Aula e Materiais da Disciplina de Programação Orientada a Objetos II (Aulas 01 a 04)*. Picos, PI: Universidade Federal do Piauí, 2026.

- [4] SOCIEDADE BRASILEIRA DE COMPUTACAO (SBC). *Modelos para Publicacao de Artigos*. Porto Alegre, RS: SBC, 2026. Disponivel em: <https://www.sbc.org.br/>.